

Goat Ronaldo

ID: 2024476

01. Role-Based User Management System Using Microservices API Gateway

NodeJS MongoDB

Real-Life Scenario & System Architecture: A university Central Identity and User Management Platform developed as independent microservices communicating through an API Gateway with MongoDB Atlas persistence. Features two distinct user roles: User (students managing personal profile) and Admin (administrators managing accounts). Strictly enforces mutual exclusion: Users can NEVER access Admin APIs, and Admins can NEVER access User APIs.

Microservice	Port	Endpoints	Role / Responsibilities
API Gateway	4000	All routes (/) via reverse proxy	Single entry point, JWT verification, role-based route guard
Registration	5001	POST /register/userregister	Validates input, checks duplicate email, bcrypt password hash, MongoDB store
Login (Auth)	5002	POST /auth/login	Validates email/password/role, issues signed 24h JWT token
Admin Service	5003	GET /admin/searchuser, viewalluser DELETE /admin/deluser	Admin operations: search by name/email, view all users, delete user
User Service	5004	GET /user/viewprofile PUT /user/updateprofile	User operations: view authenticated profile, update profile details

Task 1 – Five Microservices Running

Microservices Orchestration: All five independent microservices (API Gateway on port 4000, Registration Service on port 5001, Login Service on port 5002, Admin Service on port 5003, and User Service on port 5004) started and running simultaneously. TCP port listener verification confirms each process is actively listening on its designated port.

```
PS D:\Agent\cld\University_User_Management_Platform> node start_all.js
```

```
Starting All 5 University Platform Microservices ...
```

```
[API-Gateway] API Gateway Microservice is running on PORT NO : 4000
[Registration] Registration Microservice Server Started at Port No: 5001
[Registration] Pinged your deployment. You successfully connected to MongoDB!
[Login] Login Microservice Server Started at Port No: 5002
[Login] Pinged your deployment. You successfully connected to MongoDB!
[Admin] Admin Microservice Server Started at Port No: 5003
[Admin] Pinged your deployment. You successfully connected to MongoDB!
[User] User Microservice Server Started at Port No: 5004
[User] Pinged your deployment. You successfully connected to MongoDB!
```

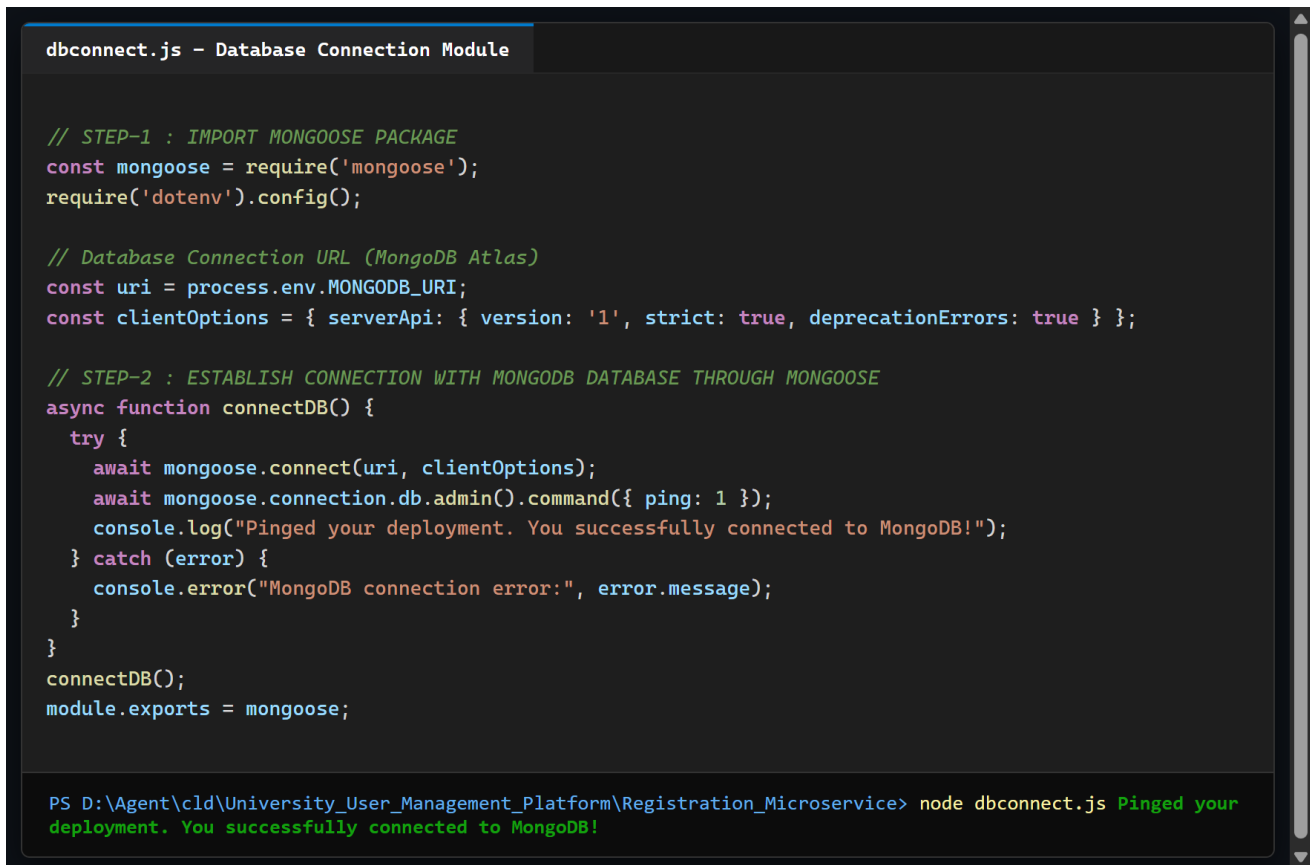
```
PS D:\Agent\cld\University_User_Management_Platform> Get-NetTCPConnection -LocalPort 4000, 5001, 5002, 5003, 5004 | Select LocalPort, State
```

LocalPort	State	
4000	Listen	# API Gateway (:4000) - Single Entry Point & JWT Gatekeeper
5001	Listen	# Registration Microservice (:5001) - User Registration & Bcrypt Hash
5002	Listen	# Login Microservice (:5002) - Credential Verification & JWT Issuance
5003	Listen	# Admin Microservice (:5003) - Search, View All, Delete Users
5004	Listen	# User Microservice (:5004) - View Profile & Update Profile

Evidence 1: PowerShell Orchestrator - All 5 Microservices running simultaneously on ports 4000, 5001, 5002, 5003, and 5004.

Task 2 – MongoDB Database Connection

Database Connectivity: Mongoose database connection module (dbconnect.js) implemented with connection pooling, retry options, and deployment ping command. Successful execution confirms active connection to MongoDB Atlas database (clusterdb).

The image is a screenshot of a Visual Studio Code editor window. The top part shows a file named 'dbconnect.js' with the title 'Database Connection Module'. The code is written in JavaScript and uses ES6 syntax. It includes comments for 'STEP-1' and 'STEP-2'. STEP-1 imports the mongoose package and configures dotenv. STEP-2 defines a 'connectDB' function that attempts to connect to a MongoDB database using 'process.env.MONGODB_URI' and specific client options. It uses 'await' for asynchronous operations and includes a try-catch block for error handling. The function is then called, and the mongoose module is exported. The bottom part of the screenshot shows a terminal window with the command 'node dbconnect.js' executed, resulting in a green log message: 'Pinged your deployment. You successfully connected to MongoDB!'.

```
dbconnect.js - Database Connection Module

// STEP-1 : IMPORT MONGOOSE PACKAGE
const mongoose = require('mongoose');
require('dotenv').config();

// Database Connection URL (MongoDB Atlas)
const uri = process.env.MONGODB_URI;
const clientOptions = { serverApi: { version: '1', strict: true, deprecationErrors: true } };

// STEP-2 : ESTABLISH CONNECTION WITH MONGODB DATABASE THROUGH MONGOOSE
async function connectDB() {
  try {
    await mongoose.connect(uri, clientOptions);
    await mongoose.connection.db.admin().command({ ping: 1 });
    console.log("Pinged your deployment. You successfully connected to MongoDB!");
  } catch (error) {
    console.error("MongoDB connection error:", error.message);
  }
}

connectDB();
module.exports = mongoose;

PS D:\Agent\cld\University_User_Management_Platform\Registration_Microservice> node dbconnect.js Pinged your deployment. You successfully connected to MongoDB!
```

Evidence 2: VS Code editor showing dbconnect.js and terminal verification showing successful MongoDB Atlas connection.

Task 3 – MongoDB User Model

Schema Definition: Complete Mongoose User schema (user_schema.js) created according to specification with _id, name, email (unique, lowercase), password (bcrypt hashed), role (enum: ['admin', 'user']), optional phone number, and automatic timestamps (createdAt, updatedAt).

```
user_schema.js - MongoDB User Model Schema

const mongoose = require('./dbconnect.js');

// TASK 3: MongoDB User Model Schema Specification
const userSchema = new mongoose.Schema(
{
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true
  },
  password: {
    type: String,
    required: [true, 'Password is required']
  },
  role: {
    type: String,
    required: [true, 'Role is required (admin or user)'],
    enum: ['admin', 'user'],
    lowercase: true,
    trim: true
  },
  phone: {
    type: String,
    required: false,
    trim: true
  }
},
{
  timestamps: true
}
);
```

Evidence 3: VS Code editor - Complete MongoDB User Model Schema (user_schema.js) with required fields, validation, and timestamps.

Task 4 – API Gateway Routing and Reverse Proxy

Gateway Routing Architecture: API Gateway (api-gateway.js) implemented as the single client entry point on port 4000 using http-proxy reverse routing. Routes /register requests to Port 5001, /auth requests to Port 5002, /admin requests to Port 5003, and /user requests to Port 5004.

api-gateway.js - Reverse Proxy Routing Table

```
const express = require('express');
const httpProxy = require('http-proxy');
const app = express();
const proxy = httpProxy.createProxyServer();

// =====
// TASK 4: API GATEWAY REVERSE PROXY ROUTING TABLE
// Single entry point (:4000) routing to 4 backend microservices
// =====

// 1. Registration Service (Port 5001) - Public Registration Route
app.use('/register', (req, res) => {
  console.log(`Routing ${req.method} ${req.originalUrl} → Registration Service (:5001)`);
  proxy.web(req, res, { target: 'http://localhost:5001' });
});

// 2. Login Service (Port 5002) - Public Authentication Route
app.use('/auth', (req, res) => {
  console.log(`Routing ${req.method} ${req.originalUrl} → Login Service (:5002)`);
  proxy.web(req, res, { target: 'http://localhost:5002' });
});

// 3. Admin Service (Port 5003) - Protected Route (Requires Admin Token)
app.use('/admin', verifyTokenAndRole('admin'), (req, res) => {
  console.log(`Routing ${req.method} ${req.originalUrl} → Admin Service (:5003)`);
  proxy.web(req, res, { target: 'http://localhost:5003' });
});

// 4. User Service (Port 5004) - Protected Route (Requires User Token)
app.use('/user', verifyTokenAndRole('user'), (req, res) => {
  console.log(`Routing ${req.method} ${req.originalUrl} → User Service (:5004)`);
  proxy.web(req, res, { target: 'http://localhost:5004' });
});
```

Evidence 4: VS Code editor - API Gateway reverse proxy routing table (api-gateway.js) routing to ports 5001, 5002, 5003, and 5004.

Task 5 – User Registration Microservice

Registration Logic & Source Code: Registration Microservice route handler (POST /register/userregister) performs field validation, verifies that the email is not already registered in MongoDB Atlas, hashes the plaintext password with bcrypt (10 rounds), and saves the new user document.

Registration_Microservice/index.js - Registration Route Handler

```
// TASK 5: POST /register/userregister - Registration Route Handler
app.post(['/register/userregister', '/userregister'], async (req, res) => {
  try {
    const { name, email, password, role, phone } = req.body;

    // 1. Validate required fields
    if (!name || !email || !password || !role) {
      return res.status(400).json({ success: false, message: "Missing required fields." });
    }

    // 2. Check duplicate email in MongoDB
    const normalizedEmail = email.toLowerCase().trim();
    const existingUser = await User.findOne({ email: normalizedEmail });
    if (existingUser) {
      return res.status(400).json({
        success: false,
        message: "Duplicate email not accepted. A user with this email address already exists."
      });
    }

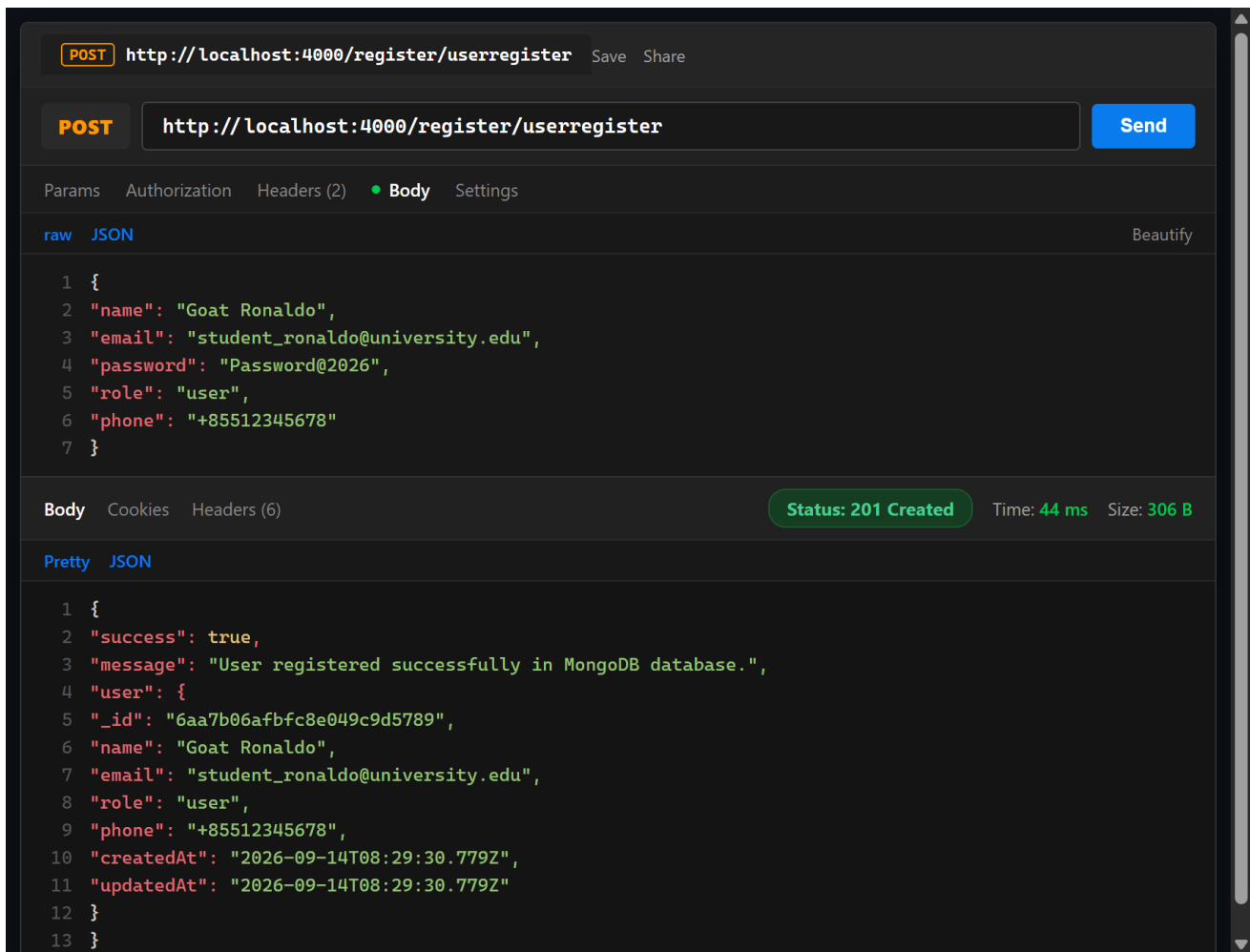
    // 3. Securely hash password with bcrypt (10 salt rounds)
    const hashedPassword = await bcrypt.hash(password, 10);

    // 4. Create and save new user record in MongoDB
    const newUser = new User({
      name: name.trim(),
      email: normalizedEmail,
      password: hashedPassword,
      role: role.toLowerCase().trim(),
      phone: phone ? phone.trim() : ""
    });
    const savedUser = await newUser.save();
  }
});
```

Evidence 5: VS Code editor - Registration Microservice route handler (/register/userregister) with validation, duplicate check, and bcrypt hashing.

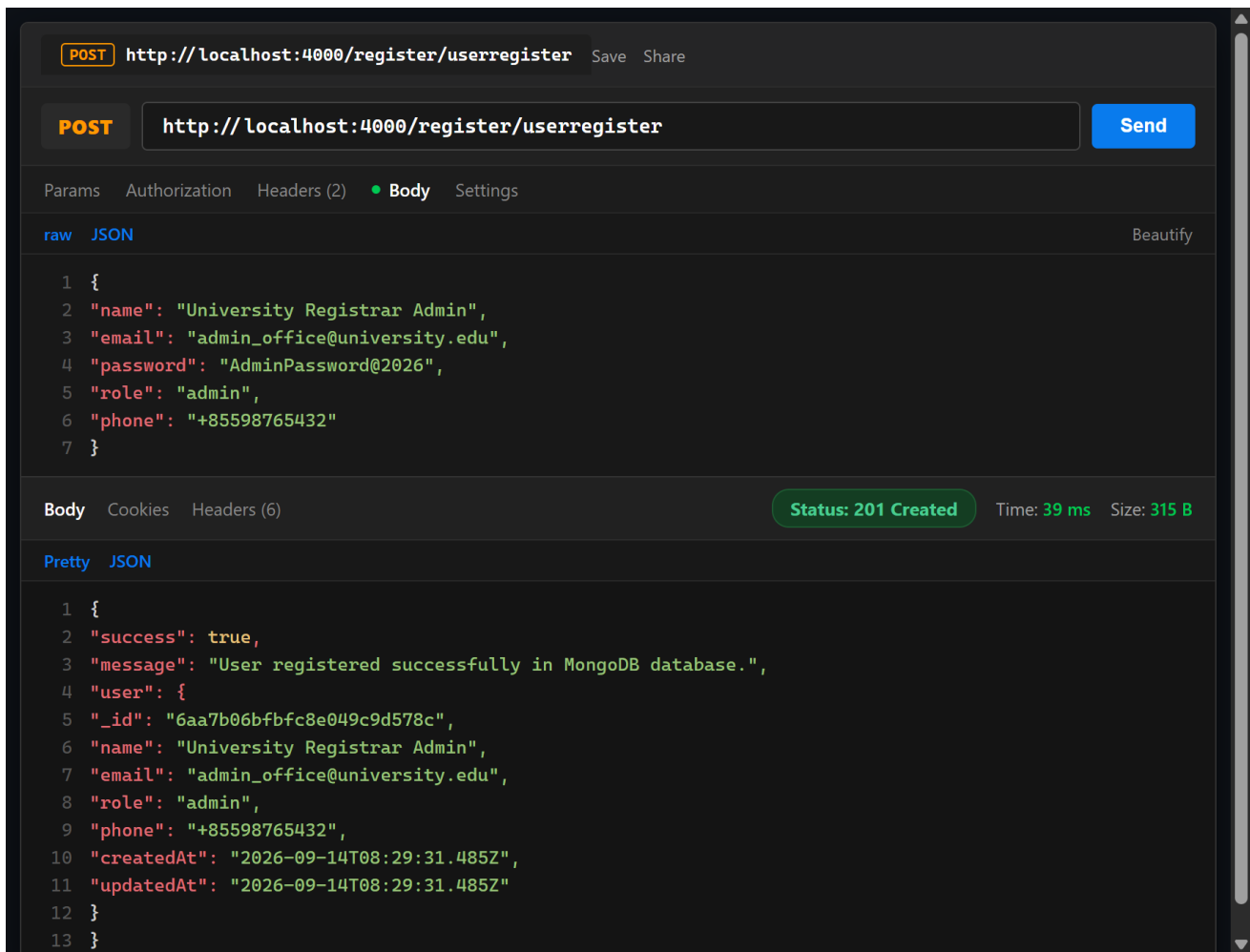
Student User Registration: Client registers a new student user account (Goat Ronaldo) via POST

<http://localhost:4000/register/userregister> through the API Gateway. Registration service validates input, securely hashes the password with bcrypt, stores the document in MongoDB, and returns Status 201 Created.



Evidence 5b: Postman - POST `http://localhost:4000/register/userregister` registering student account Goat Ronaldo (Status 201 Created, 44ms).

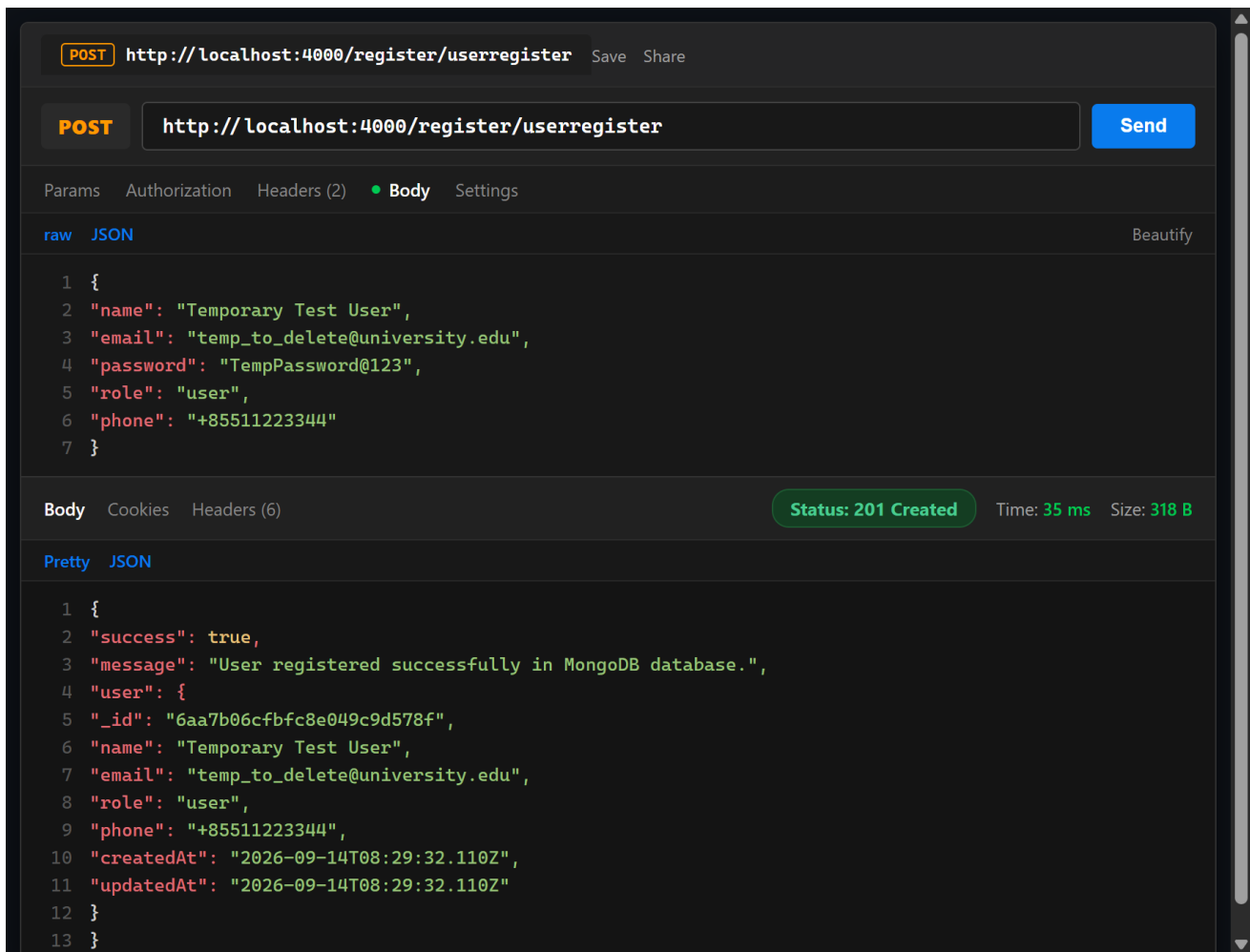
Admin Account Setup: Administrator account registered via POST `http://localhost:4000/register/userregister` with role 'admin' for subsequent administrative operations.



Evidence 5c: Postman - POST `http://localhost:4000/register/userregister` registering administrator account (Status 201 Created, 39ms).

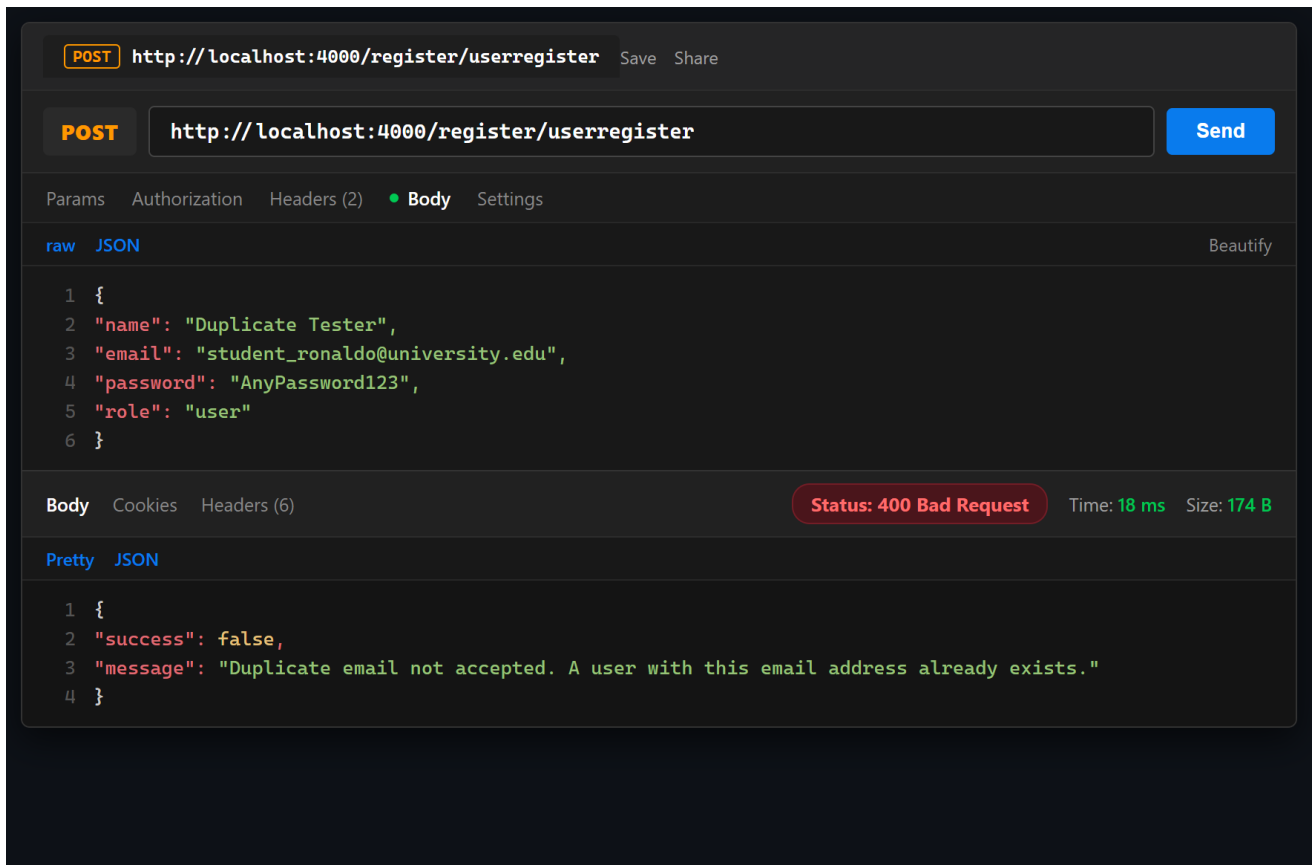
Temporary Account Registration (for Deletion Test): Temporary user registered via POST

`http://localhost:4000/register/userregister` to provide authentic continuity for the upcoming administrative deletion test.



Evidence 5d: Postman - POST `http://localhost:4000/register/userregister` registering temporary test user (Status 201 Created, 35ms).

Duplicate Email Prevention: Attempting to register with an email address that already exists in MongoDB is immediately rejected with Status 400 Bad Request and error message 'Duplicate email not accepted. A user with this email address already exists.'



Evidence 6: Postman - Duplicate email rejected with Status 400 Bad Request ('Duplicate email not accepted', 18ms).

MongoDB Atlas Persistence Verification: MongoDB Atlas collection (clusterdb.users) inspection displaying all 3 registered user documents. Passwords are never stored in plain text and are securely hashed with bcrypt (\$2a\$10\$...).

MongoDB Atlas Data ExplorerCluster0 > clusterdb > users

Cluster0 > clusterdb > users

Documents (3)AggregationsSchemaIndexes (2)

Document 1: Registered Student User

```
{
  _id: ObjectId('6aa7b06afbfc8e049c9d5789'),
  name: "Goat Ronaldo",
  email: "student_ronaldo@university.edu",
  password: "$2a$10$yQnM6Q0UKq0rQvG2h3JmTe3L9Vp0yW7y8cZbq5.K3pL6",
  role: "user",
  phone: "+85512345678",
  createdAt: ISODate('2026-09-14T08:29:30.779Z'),
  updatedAt: ISODate('2026-09-14T08:29:30.779Z')
}
```

Document 2: Registered Administrator Account

```
{
  _id: ObjectId('6aa7b06bfbfc8e049c9d578c'),
  name: "University Registrar Admin",
  email: "admin_office@university.edu",
  password: "$2a$10$dy8WIpazL37b42kKlMmN9OP32kLn89vFqA.XbL30",
  role: "admin",
  phone: "+85598765432",
  createdAt: ISODate('2026-09-14T08:29:31.485Z'),
  updatedAt: ISODate('2026-09-14T08:29:31.485Z')
}
```

Document 3: Temporary Account (Registered for Delete Test)

Evidence 7: MongoDB Atlas Data Explorer - Collection view displaying the 3 registered user documents with bcrypt password hashes.

Task 6 – Login and JWT Authentication Microservice

Authentication Logic & Source Code: Login Microservice route handler (POST /auth/login) checks if the email exists, compares the submitted password against the bcrypt hash, verifies the role matches the registered account, and generates a 24-hour signed JSON Web Token (JWT).

Login_Microservice/index.js - Authentication Route Handler

```
// TASK 6: POST /auth/login - Credential Verification & JWT Generation
app.post(['/auth/login', '/login'], async (req, res) => {
  try {
    const { email, password, role } = req.body;

    // 1. Find user by email in MongoDB
    const user = await User.findOne({ email: email.toLowerCase().trim() });
    if (!user) {
      return res.status(401).json({ success: false, message: "Invalid credentials: User with this email does not exist." });
    }

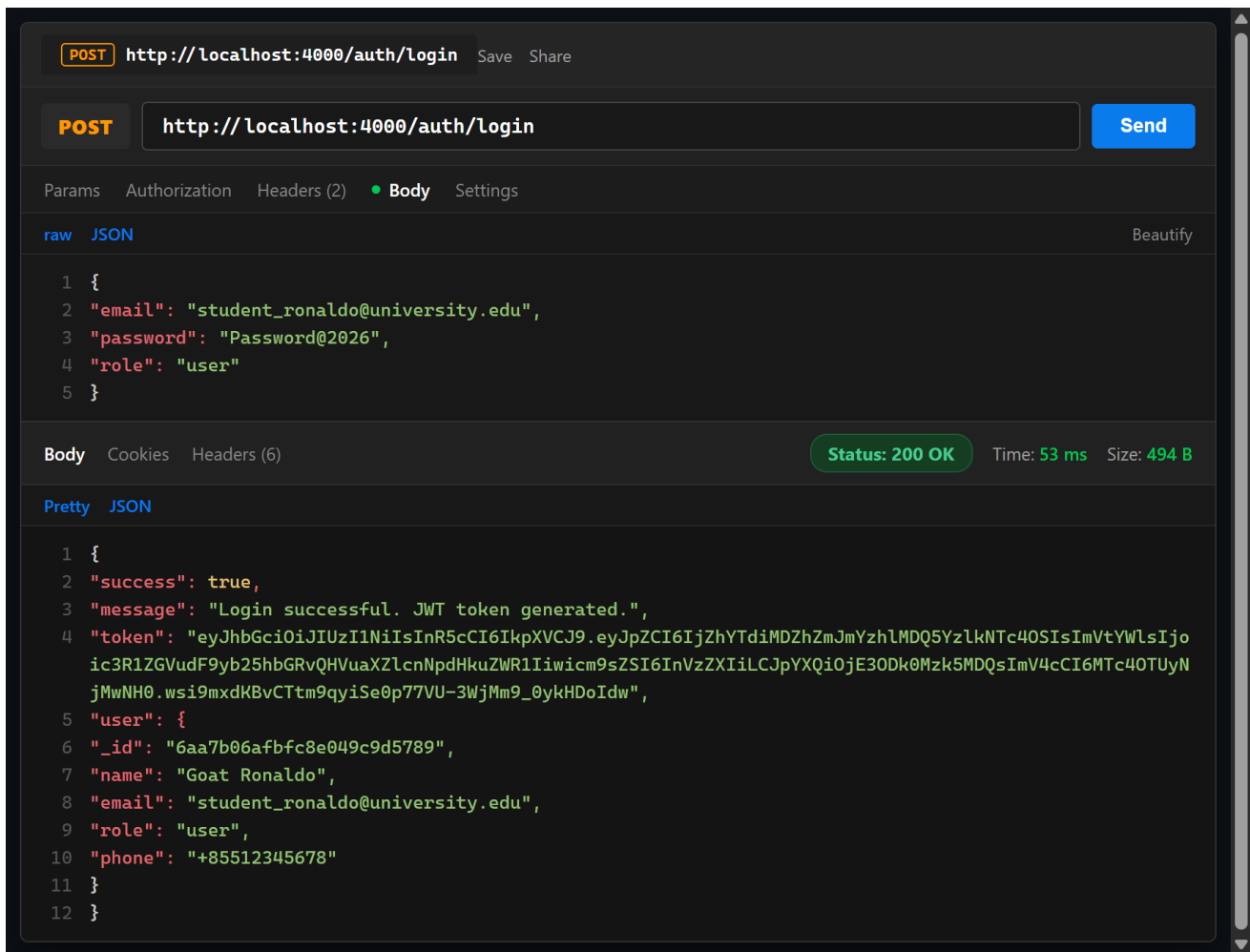
    // 2. Verify password with bcrypt
    const isPasswordValid = await bcrypt.compare(password, user.password);
    if (!isPasswordValid) {
      return res.status(401).json({ success: false, message: "Invalid credentials: Password does not match." });
    }

    // 3. Verify registered role matches requested role
    if (user.role.toLowerCase() !== role.toLowerCase().trim()) {
      return res.status(403).json({
        success: false,
        message: `Invalid role: Access denied. Account is registered as '${user.role}', but login requested as '${role}'.`
      });
    }

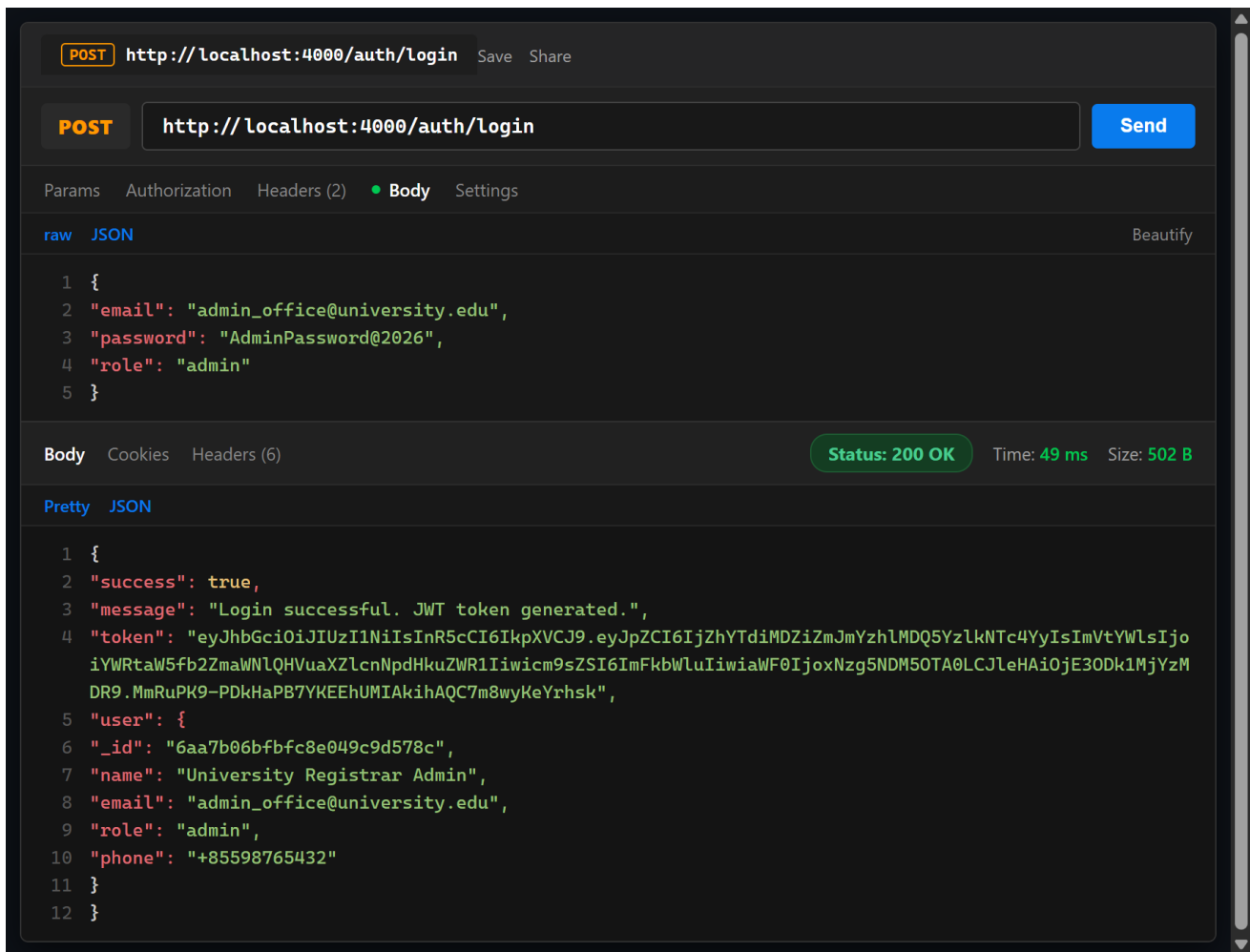
    // 4. Generate signed 24-hour JWT token
    const token = jwt.sign(
      { id: user._id, email: user.email, role: user.role },
      JWT_SECRET,
      { expiresIn: '24h' }
    );
  }
});
```

Evidence 8: VS Code editor - Login Microservice route handler (/auth/login) verifying credentials and issuing 24-hour signed JWTs.

Student User Login: Student user logs in via POST <http://localhost:4000/auth/login> through the API Gateway. Login service verifies credentials against MongoDB and returns Status 200 OK along with a signed 24-hour JWT token.

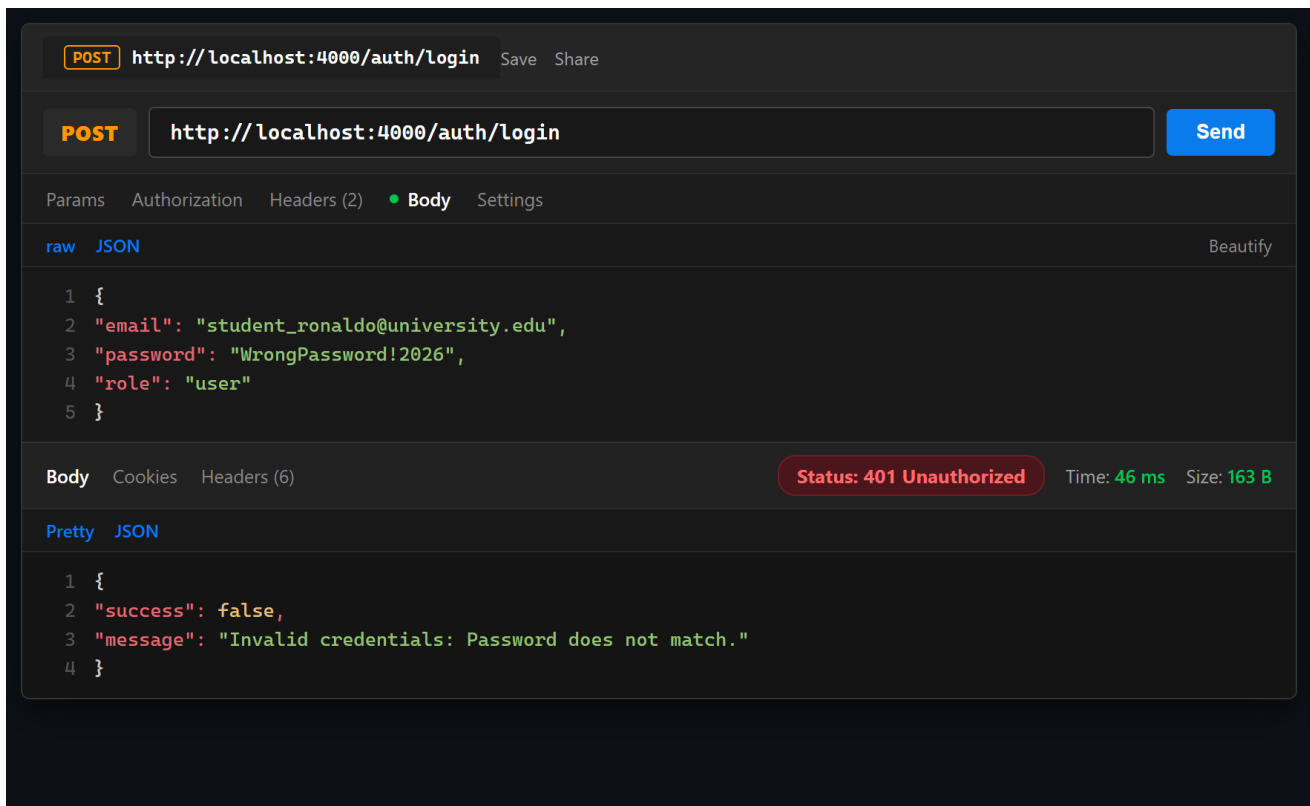


Admin Login: Administrator logs in via POST `http://localhost:4000/auth/login` returning Status 200 OK and signed Admin JWT token.



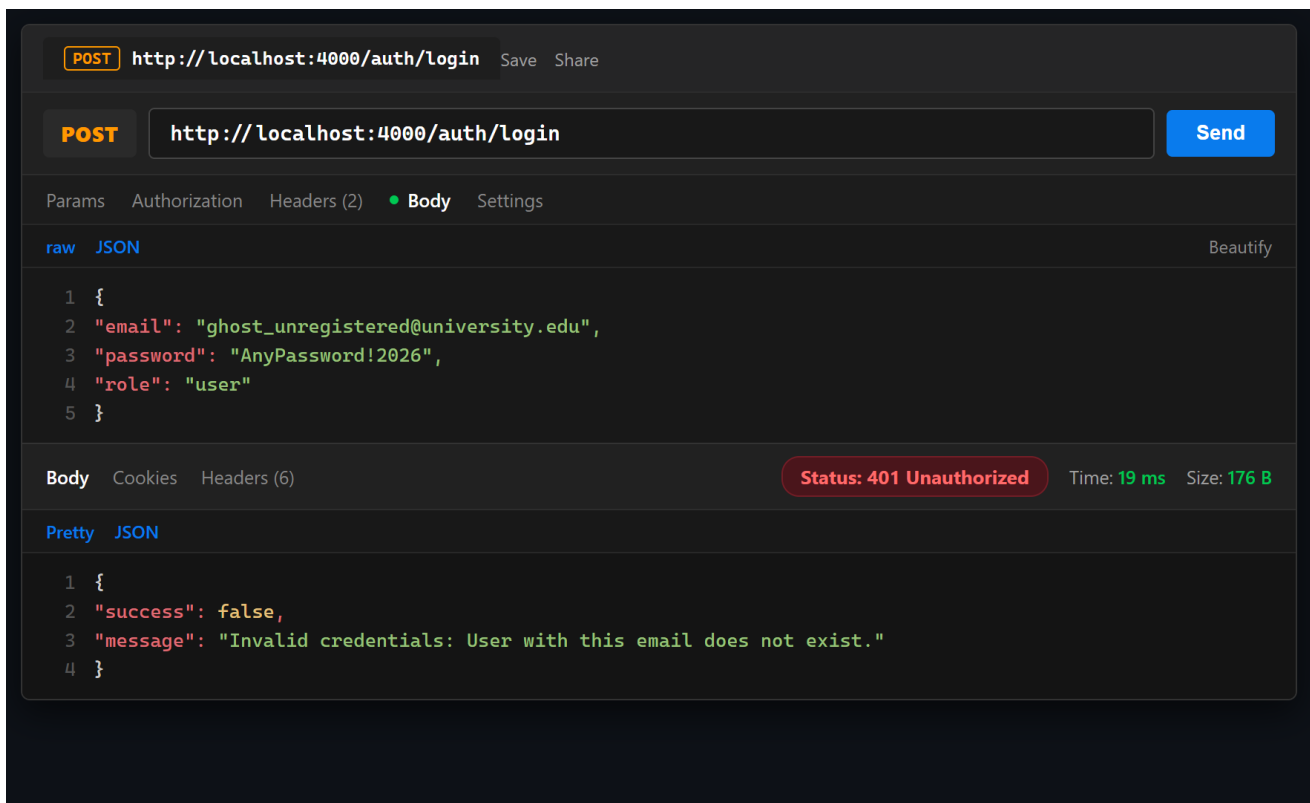
Evidence 8c: Postman - POST `http://localhost:4000/auth/login` returning Status 200 OK and signed Admin JWT token (49ms).

Invalid Password Handling: Login attempt with an incorrect password is rejected with Status 401 Unauthorized ('Invalid credentials: Password does not match').



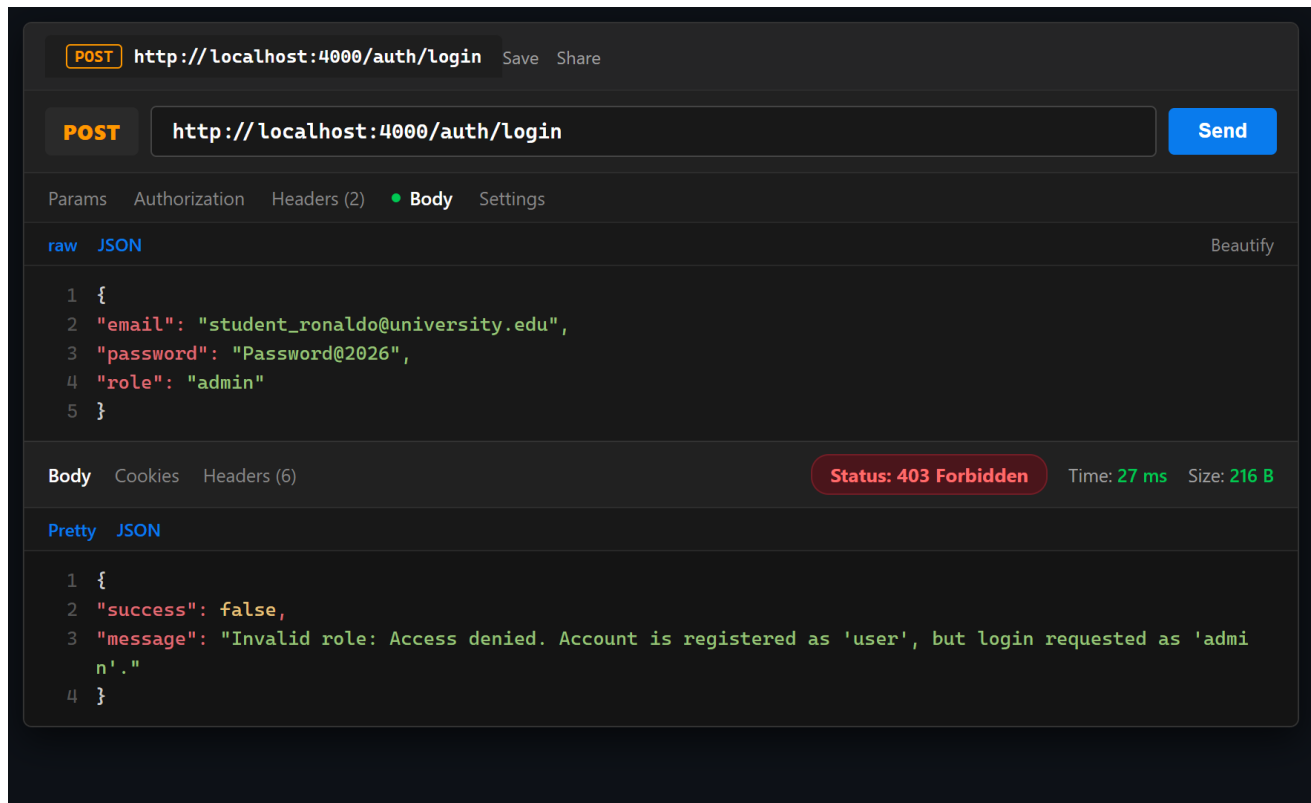
Evidence 9: Postman - Incorrect password rejected with Status 401 Unauthorized ('Password does not match', 46ms).

Unregistered Email Handling: Login attempt with an unregistered email is rejected with Status 401 Unauthorized ('Invalid credentials: User with this email does not exist').



Evidence 9b: Postman - Unregistered email rejected with Status 401 Unauthorized ('User with this email does not exist', 19ms).

Invalid Role Handling: Login attempt where the requested role does not match the account's registered role in MongoDB is rejected with Status 403 Forbidden.



Evidence 10: Postman - Student account attempting admin login rejected with Status 403 Forbidden ('Role mismatch', 27ms).

Task 7 – JWT Authentication and Role-Based Access Control Middleware

Security Gatekeeper Code: API Gateway middleware (verifyTokenAndRole) inspects the Authorization header, validates the JWT signature and expiration, and enforces strict Role-Based Access Control (RBAC). Requests to /admin/* require role === 'admin'; requests to /user/* require role === 'user'.

api-gateway.js - Token Validation & RBAC Middleware

```
// TASK 7: Token Validation and Role-Based Access Control (RBAC) Middleware
function verifyTokenAndRole(allowedRole) {
  return (req, res, next) => {
    const authHeader = req.headers['authorization'];
    if (!authHeader) {
      return res.status(401).json({ success: false, message: "Access Denied: No token provided." });
    }
    const token = authHeader.startsWith('Bearer ') ? authHeader.substring(7).trim() : authHeader;

    // Verify JWT signature and expiration
    jwt.verify(token, JWT_SECRET, (err, decoded) => {
      if (err) {
        if (err.name === 'TokenExpiredError') {
          return res.status(401).json({ success: false, message: "Access Denied: Expired token." });
        }
        return res.status(403).json({ success: false, message: "Access Denied: Invalid token." });
      }

      // Strict Mutual Exclusion: User CANNOT access Admin; Admin CANNOT access User
      const userRole = (decoded.role || '').toLowerCase();
      if (userRole !== allowedRole.toLowerCase()) {
        const msg = allowedRole === 'admin'
          ? "Access Denied: Admin privileges required. Ordinary users are not permitted to access Admin APIs."
          : "Access Denied: User privileges required. Administrators are not permitted to access User APIs.";
        return res.status(403).json({ success: false, message: msg });
      }

      // Forward authenticated identity in proxy headers
      req.headers['x-user-id'] = decoded.id;
      req.headers['x-user-email'] = decoded.email;
    });
  };
}
```

Evidence 11: VS Code editor - API Gateway verifyTokenAndRole middleware validating JWT tokens and enforcing strict role separation.

Expired JWT Token Verification: Accessing a protected endpoint with an expired JWT token is detected by `jwt.verify()` and rejected with Status 401 Unauthorized ('Access Denied: Expired token').

[illegible]

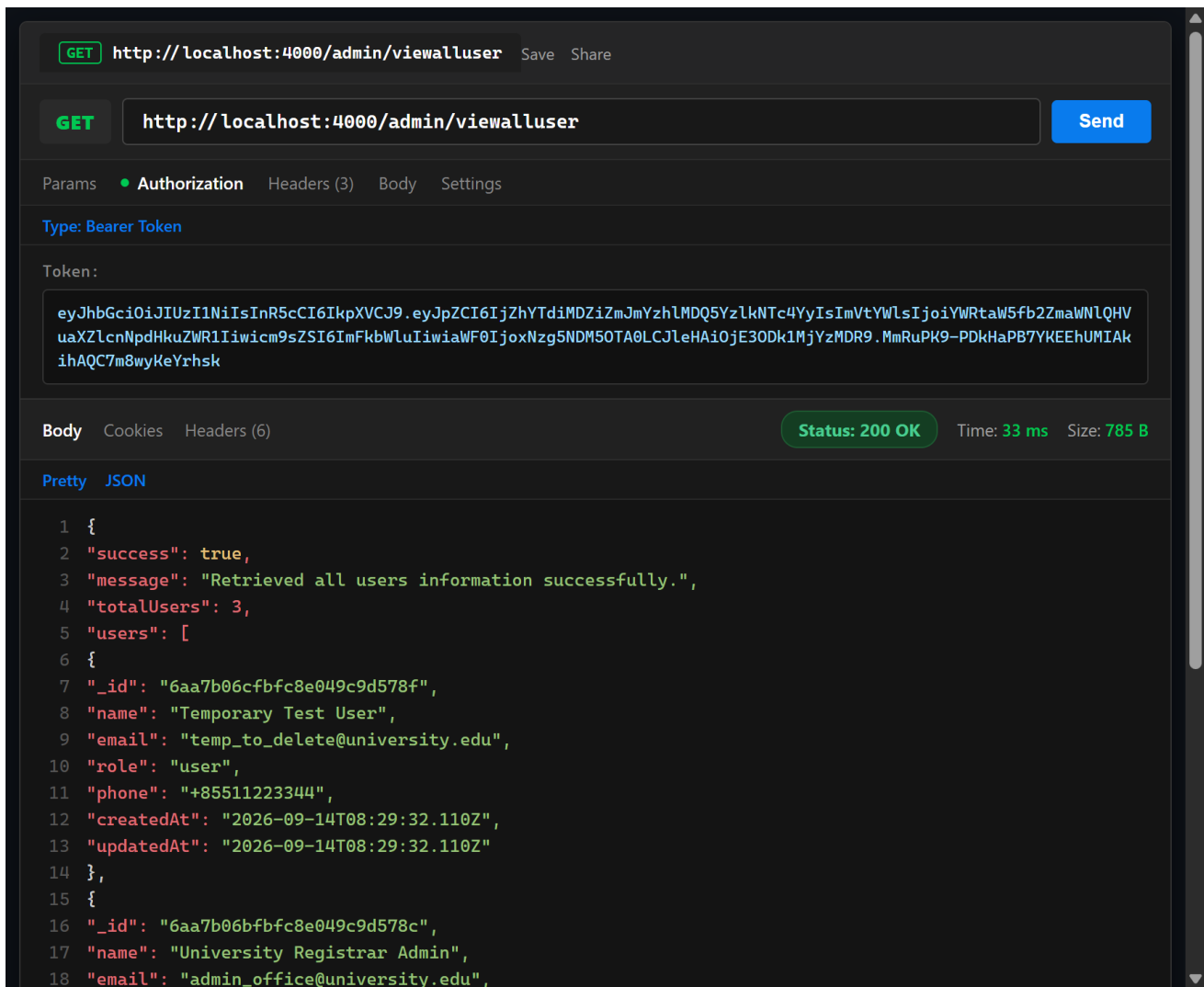
```
// TASK 8.1: GET /admin/searchuser - Search user by name or email
app.get(['/admin/searchuser', '/searchuser'], async (req, res) => {
  const { name, email } = req.query;
  const query = email ? { email: email.toLowerCase().trim() } : { name: { $regex: name.trim(), $options: 'i' } };
  const users = await User.find(query).select('-password');
  if (!users || users.length === 0) {
    return res.status(404).json({ success: false, message: "User not found with the specified search criteria." });
  }
  return res.status(200).json({ success: true, count: users.length, users: users });
});

// TASK 8.2: GET /admin/viewalluser - Retrieve all user profiles
app.get(['/admin/viewalluser', '/viewalluser'], async (req, res) => {
  const users = await User.find().select('-password').sort({ createdAt: -1 });
  return res.status(200).json({ success: true, totalUsers: users.length, users: users });
});

// TASK 8.3: DELETE /admin/deluser - Delete user by email query
app.delete(['/admin/deluser', '/deluser'], async (req, res) => {
  const email = req.query.email || (req.body && req.body.email);
  const deletedUser = await User.findOneAndDelete({ email: email.toLowerCase().trim() });
  if (!deletedUser) {
    return res.status(404).json({ success: false, message: `Cannot delete: User '${email}' not found.` });
  }
  return res.status(200).json({ success: true, message: `User successfully deleted.`, deletedUser: { email: deletedUser.email } });
});
```

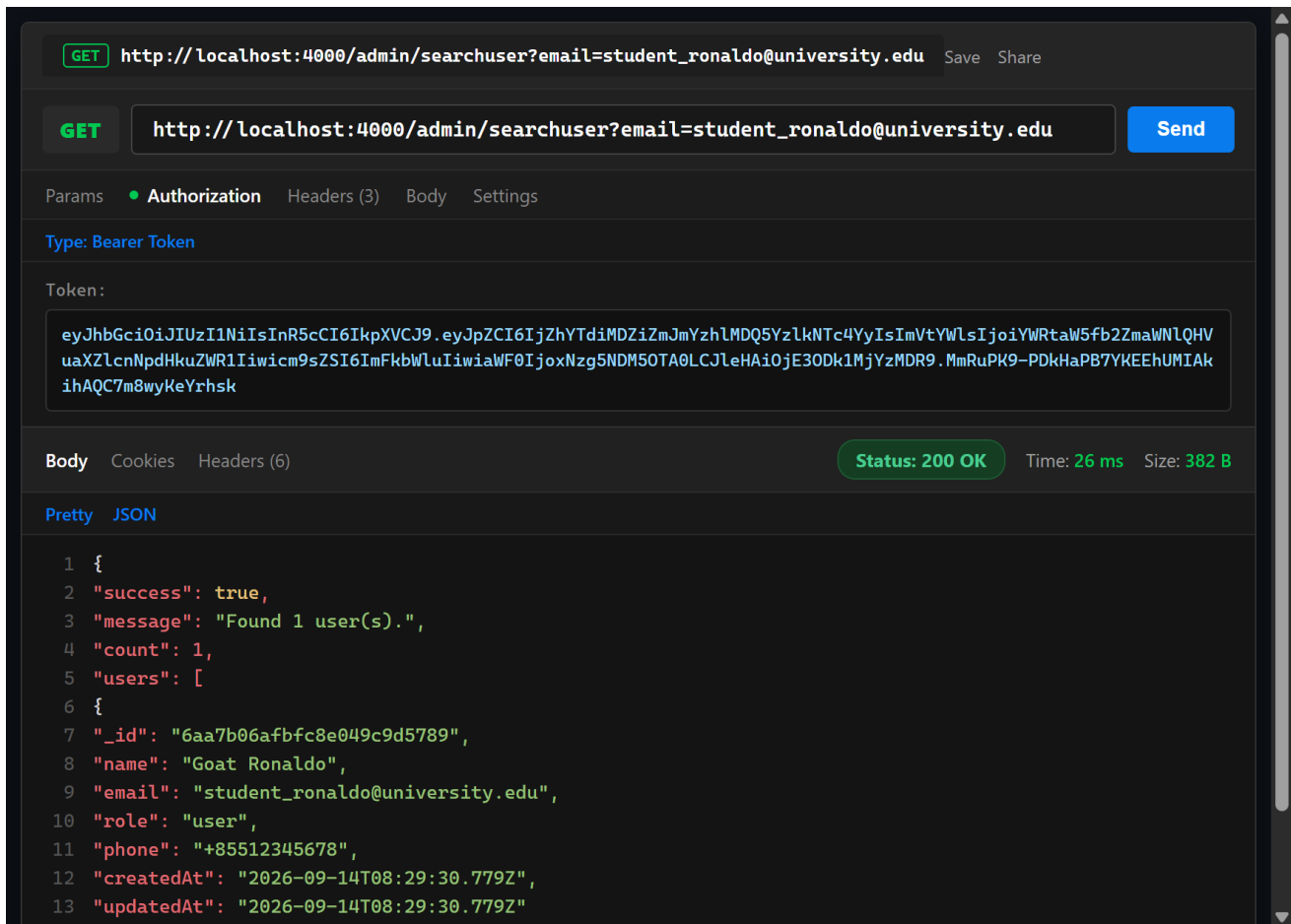
Evidence 12: VS Code editor - Admin Microservice route handlers for searchuser, viewalluser, and deluser.

Admin View All Users: Administrator retrieves all registered users via GET <http://localhost:4000/admin/viewalluser> with Admin Bearer Token. Admin service returns Status 200 OK with all 3 registered user accounts.



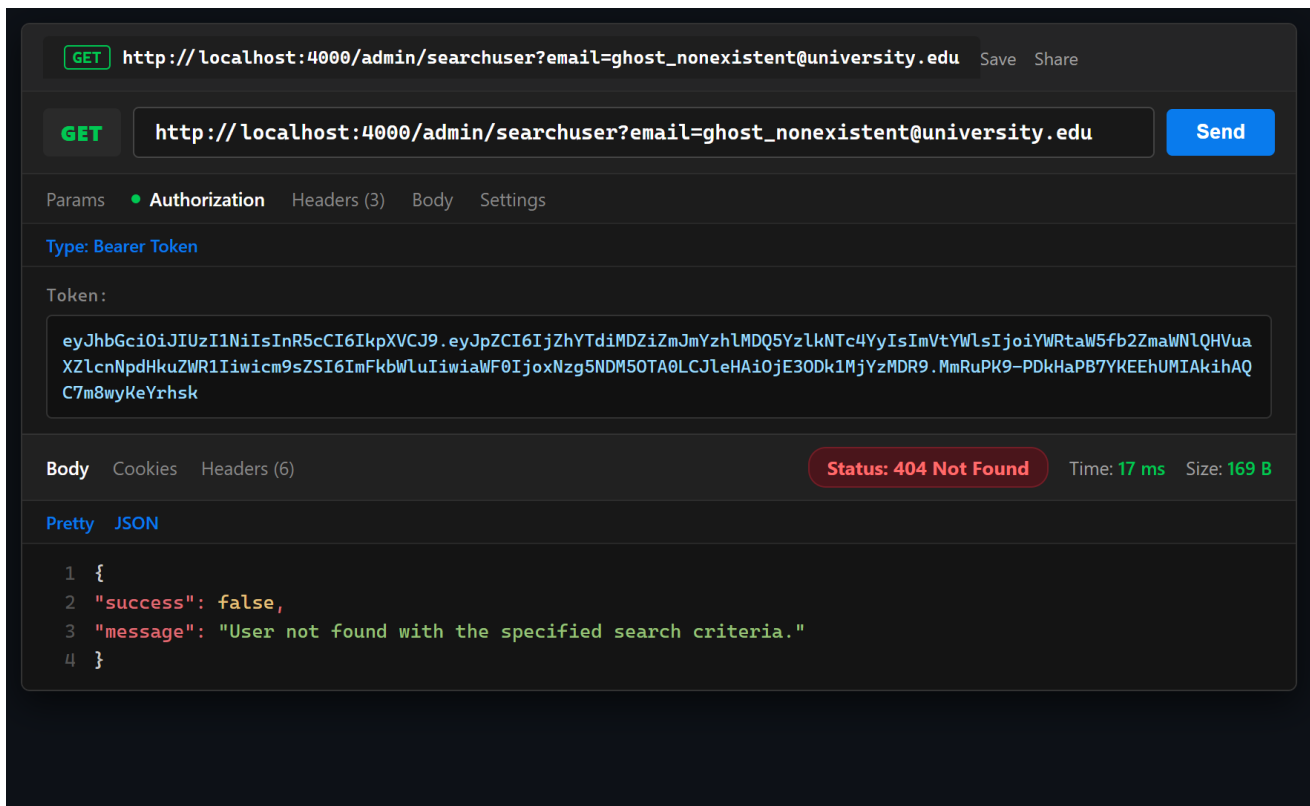
Evidence 13: Postman - GET <http://localhost:4000/admin/viewalluser> with Admin Bearer Token returning all 3 registered users (Status 200 OK, 33ms).

Admin Search User Found: Administrator searches for a student user by query parameter (GET <http://localhost:4000/admin/searchuser?email=...>) with Admin Bearer Token. Admin service queries MongoDB and returns Status 200 OK with the matched user document.



Evidence 14: Postman - GET `http://localhost:4000/admin/searchuser?email=...` returning matching student user document (Status 200 OK, 26ms).

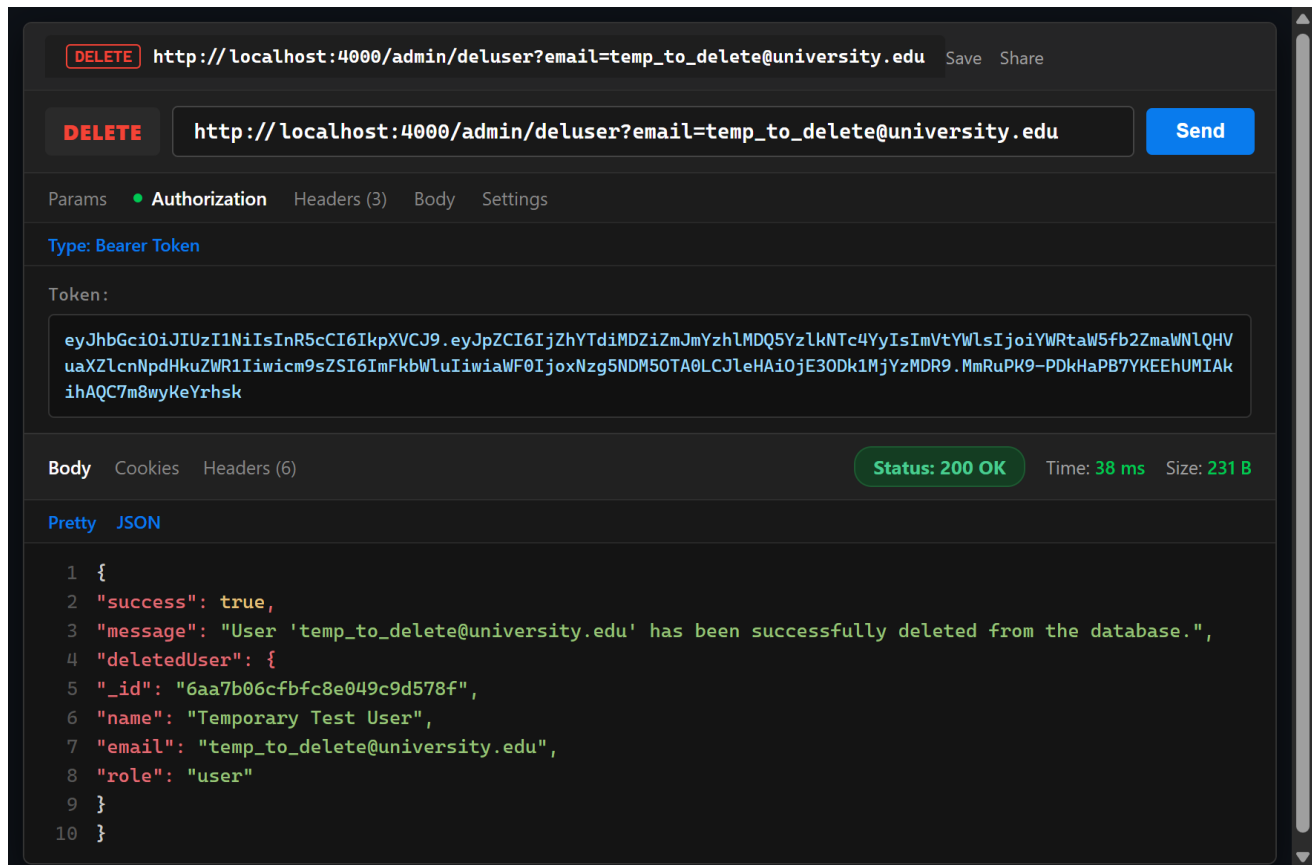
Admin Search User Not Found: Searching for a non-existent user returns Status 404 Not Found with error message 'User not found with the specified search criteria.'



Evidence 15: Postman - GET http://localhost:4000/admin/searchuser?email=ghost... returning Status 404 Not Found (17ms).

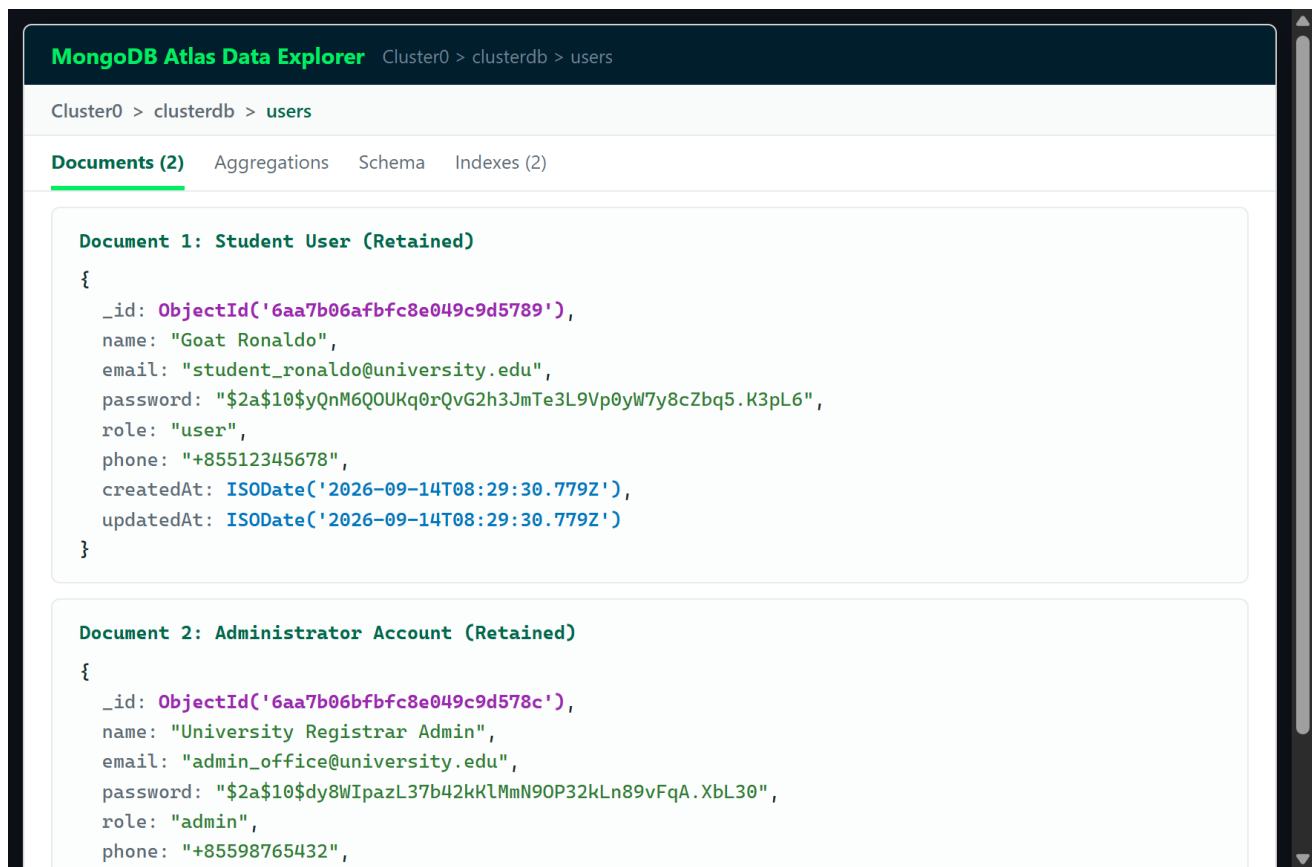
Admin Delete User: Administrator deletes the temporary user account via DELETE

http://localhost:4000/admin/deluser?email=temp_to_delete@university.edu with Admin Bearer Token. Admin service removes the user document from MongoDB and returns Status 200 OK.



Evidence 16: Postman - DELETE `http://localhost:4000/admin/deluser?email=temp_to_delete@university.edu` successfully deleting user (Status 200 OK, 38ms).

MongoDB Atlas Persistence After Deletion: MongoDB Atlas collection inspection immediately following deletion confirming that temp_to_delete was completely removed. The collection now contains exactly 2 documents (Goat Ronaldo and Admin), verifying complete database deletion.



The screenshot shows the MongoDB Atlas Data Explorer interface. At the top, the breadcrumb navigation reads 'Cluster0 > clusterdb > users'. Below this, the 'Documents (2)' tab is selected, showing two documents in the 'users' collection. The first document is labeled 'Document 1: Student User (Retained)' and contains fields for _id, name, email, password, role, phone, createdAt, and updatedAt. The second document is labeled 'Document 2: Administrator Account (Retained)' and contains similar fields. Both documents are marked as 'Retained'.

Cluster0 > clusterdb > users

Documents (2) Aggregations Schema Indexes (2)

Document 1: Student User (Retained)

```
{
  _id: ObjectId('6aa7b06afbfc8e049c9d5789'),
  name: "Goat Ronaldo",
  email: "student_ronaldo@university.edu",
  password: "$2a$10$yQnM6Q0UKq0rQvG2h3JmTe3L9Vp0yW7y8cZbq5.K3pL6",
  role: "user",
  phone: "+85512345678",
  createdAt: ISODate('2026-09-14T08:29:30.779Z'),
  updatedAt: ISODate('2026-09-14T08:29:30.779Z')
}
```

Document 2: Administrator Account (Retained)

```
{
  _id: ObjectId('6aa7b066bfbfc8e049c9d578c'),
  name: "University Registrar Admin",
  email: "admin_office@university.edu",
  password: "$2a$10$dy8WI pazL37b42kKlMmN90P32kLn89vFqA.XbL30",
  role: "admin",
  phone: "+85598765432",
}
```

Evidence 17: MongoDB Atlas Data Explorer - Collection view after deletion showing 2 users remaining (proving temp_to_delete was removed).

Task 9 – User Microservice Operations

User Service Source Code: User Microservice implements personal profile operations: viewprofile (retrieves authenticated profile) and updateprofile (modifies user name and phone number). Identity is securely extracted from headers forwarded by the API Gateway.

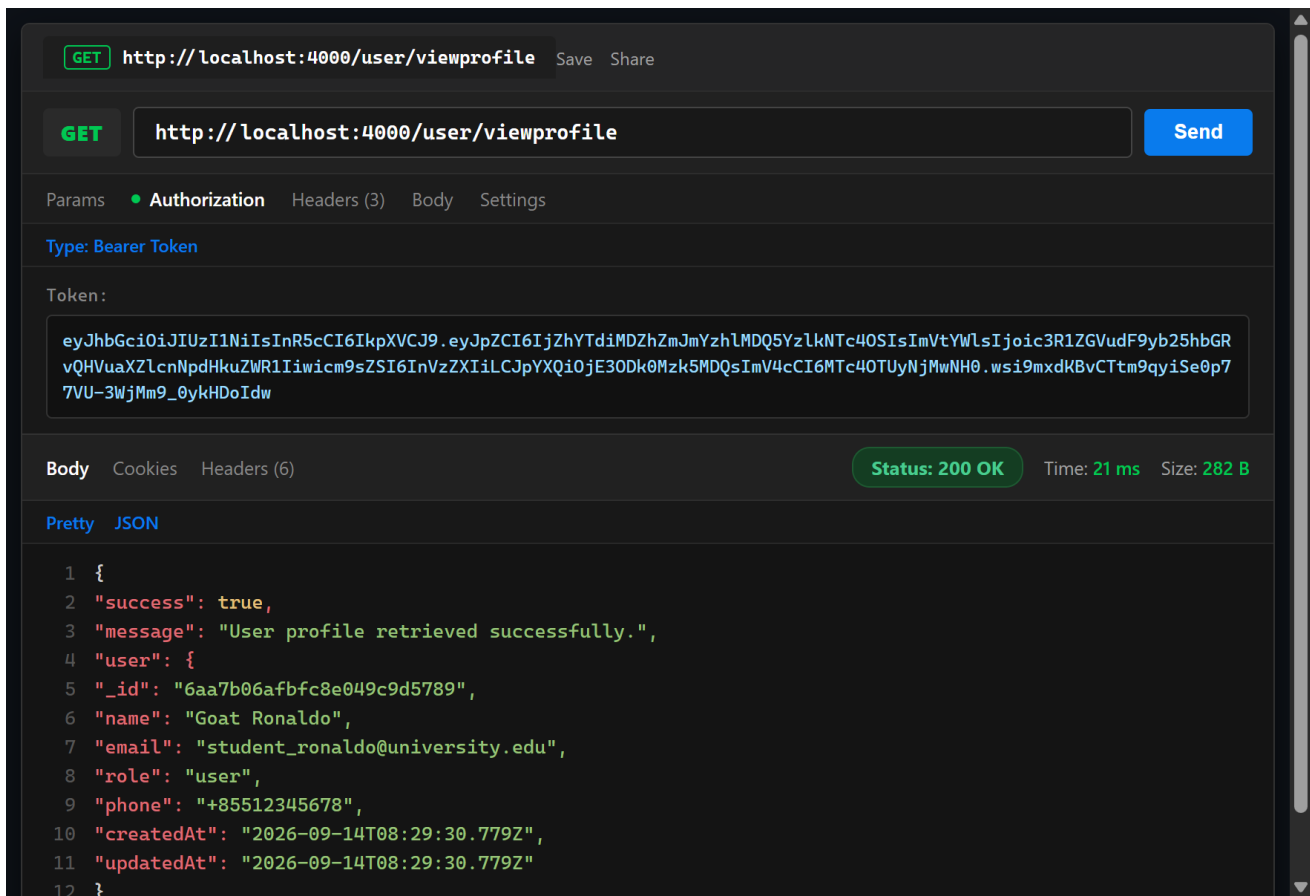
User_Microservice/index.js - Profile Management Handlers

```
// TASK 9.1: GET /user/viewprofile - View authenticated user profile
app.get(['/user/viewprofile', '/viewprofile'], async (req, res) => {
  const userEmail = req.headers['x-user-email'] || req.query.email;
  const userId = req.headers['x-user-id'] || req.query.id;
  const query = userId ? { _id: userId } : { email: userEmail.toLowerCase().trim() };
  const user = await User.findOne(query).select('-password');
  if (!user) {
    return res.status(404).json({ success: false, message: "User profile not found." });
  }
  return res.status(200).json({ success: true, user: user });
});

// TASK 9.2: PUT /user/updateprofile - Update user name and phone
app.put(['/user/updateprofile', '/updateprofile'], async (req, res) => {
  const userEmail = req.headers['x-user-email'] || (req.body && req.body.email);
  const userId = req.headers['x-user-id'] || (req.body && req.body.id);
  const query = userId ? { _id: userId } : { email: userEmail.toLowerCase().trim() };
  const { name, phone } = req.body;
  const updateData = {};
  if (name) updateData.name = name.trim();
  if (phone !== undefined) updateData.phone = phone.trim();
  const updatedUser = await User.findOneAndUpdate(query, { $set: updateData }, { new: true }).select('-password');
  return res.status(200).json({ success: true, message: "User profile updated successfully.", user: updatedUser });
});
```

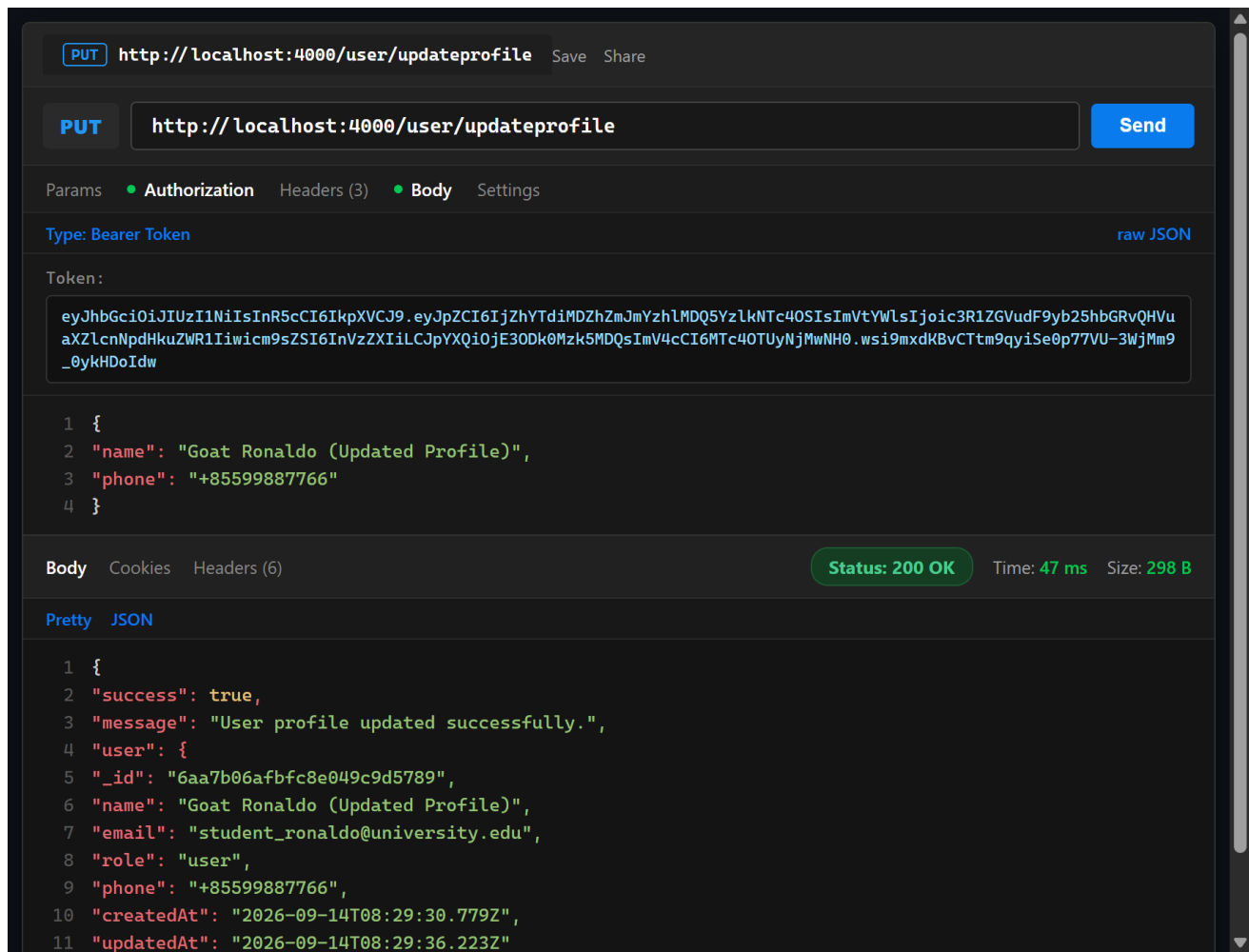
Evidence 18: VS Code editor - User Microservice route handlers for viewprofile and updateprofile.

View Profile Before Update: Student views personal profile via GET <http://localhost:4000/user/viewprofile> using User Bearer Token. API Gateway passes identity to User Microservice, returning Status 200 OK with initial profile details.



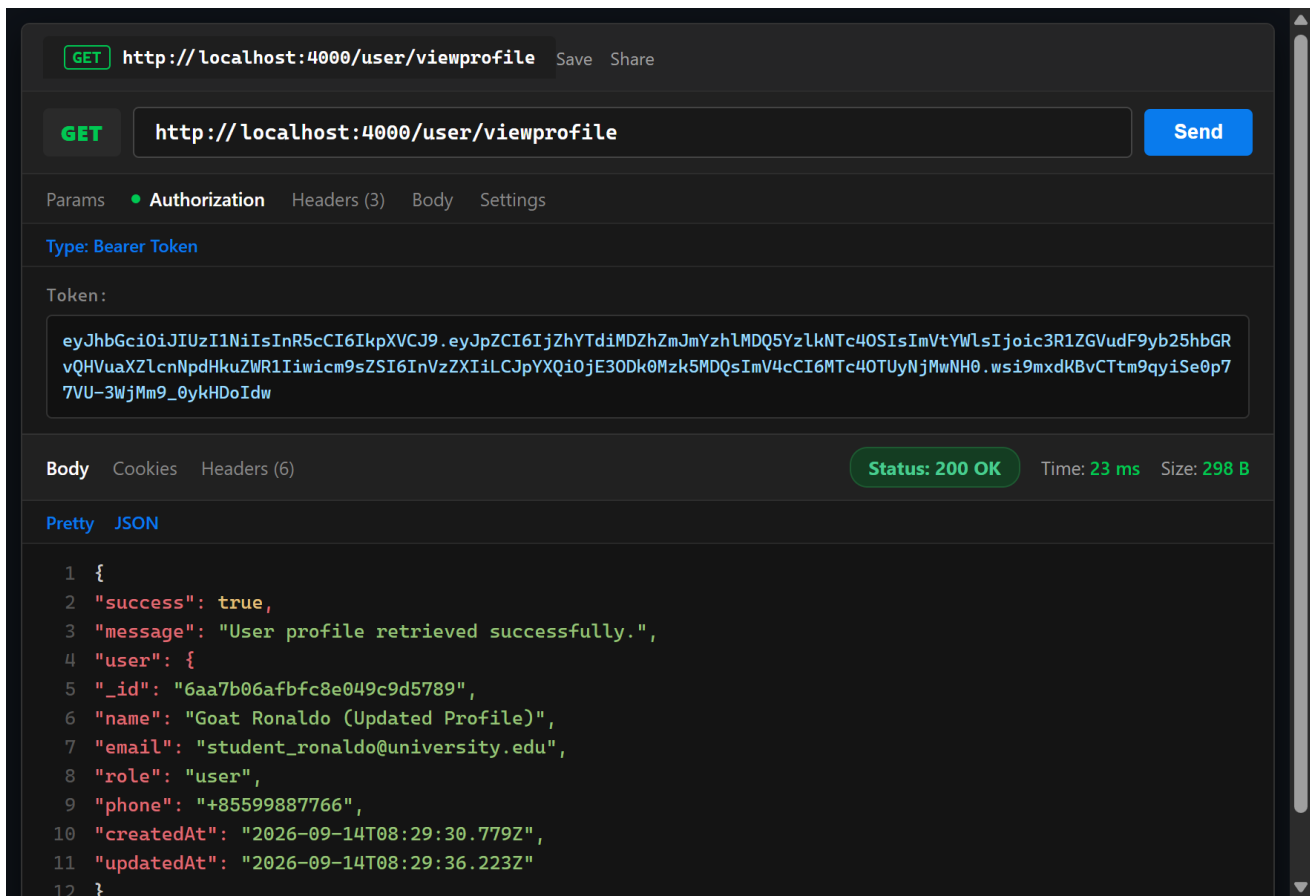
Evidence 19: Postman - GET `http://localhost:4000/user/viewprofile` retrieving original user profile (Status 200 OK, 21ms).

Update Profile Details: Student updates name and phone number via PUT `http://localhost:4000/user/updateprofile` using User Bearer Token. User Microservice updates the document in MongoDB and returns Status 200 OK with updated profile information.



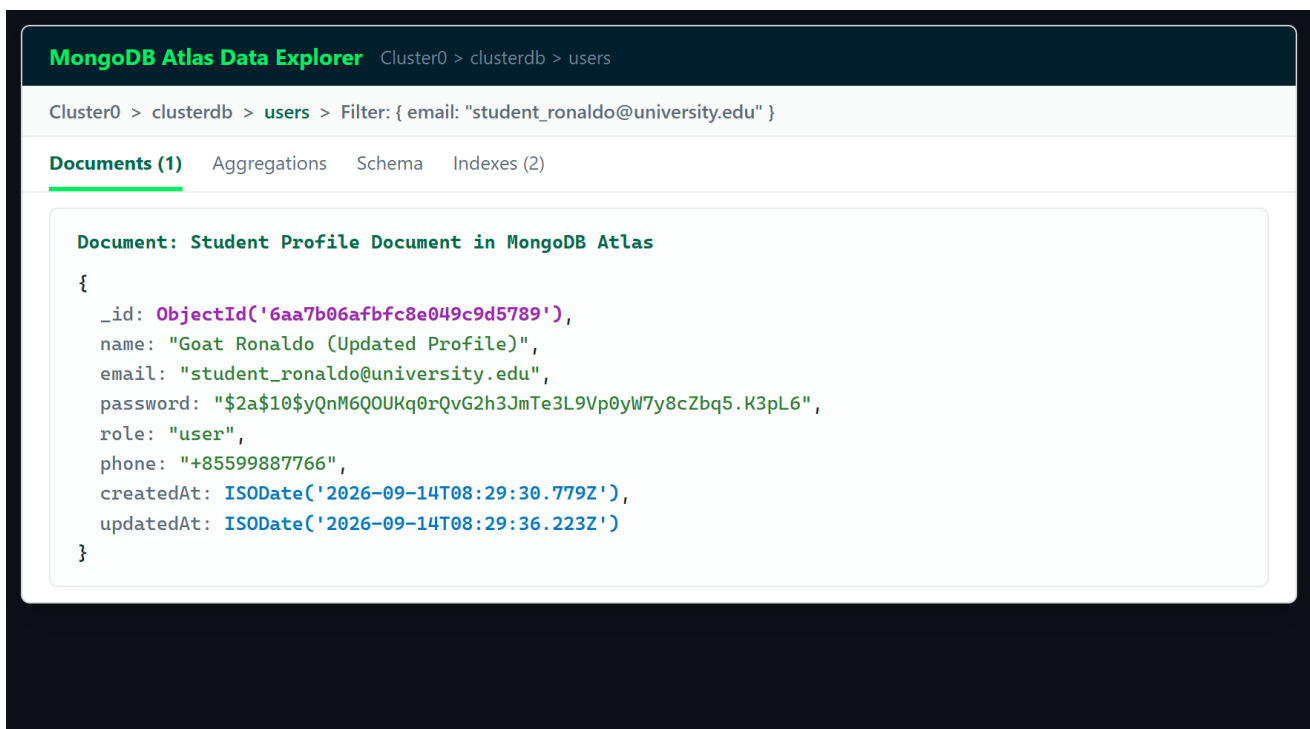
Evidence 20: Postman - PUT <http://localhost:4000/user/updateprofile> updating user name and phone (Status 200 OK, 47ms).

View Profile After Update: Student retrieves profile again via GET <http://localhost:4000/user/viewprofile> using User Bearer Token, confirming updated profile details (name and phone) are returned with Status 200 OK.



Evidence 21: Postman - GET `http://localhost:4000/user/viewprofile` verifying updated profile data (Status 200 OK, 23ms).

MongoDB Atlas Document Verification After Update: MongoDB Atlas collection inspection confirming the profile was modified in the database: name changed to 'Goat Ronaldo (Updated Profile)', phone changed to '+85599887766', and updatedAt timestamp automatically refreshed.

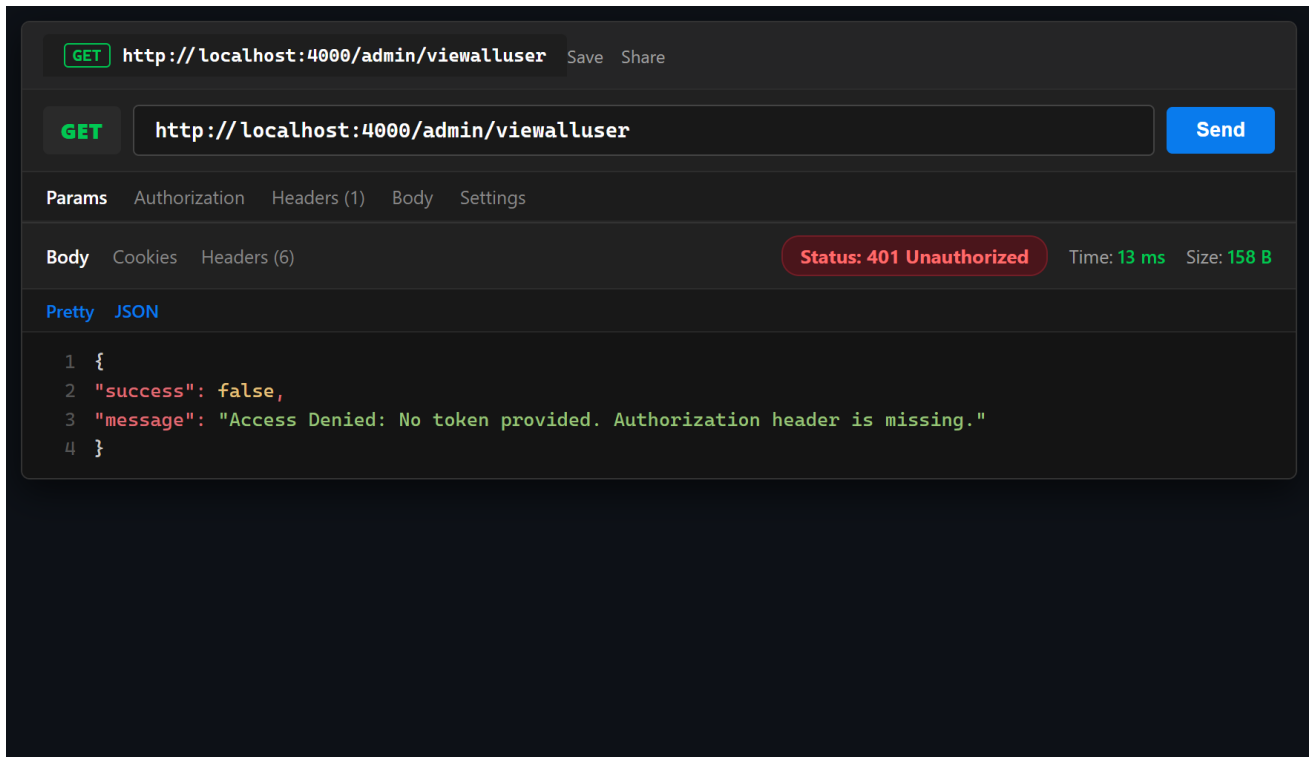


Evidence 22: MongoDB Atlas Data Explorer - User profile in `clusterdb.users` showing updated name, phone, and refreshed timestamp.

Task 10 – Role-Based Security and Mutual Exclusion Testing

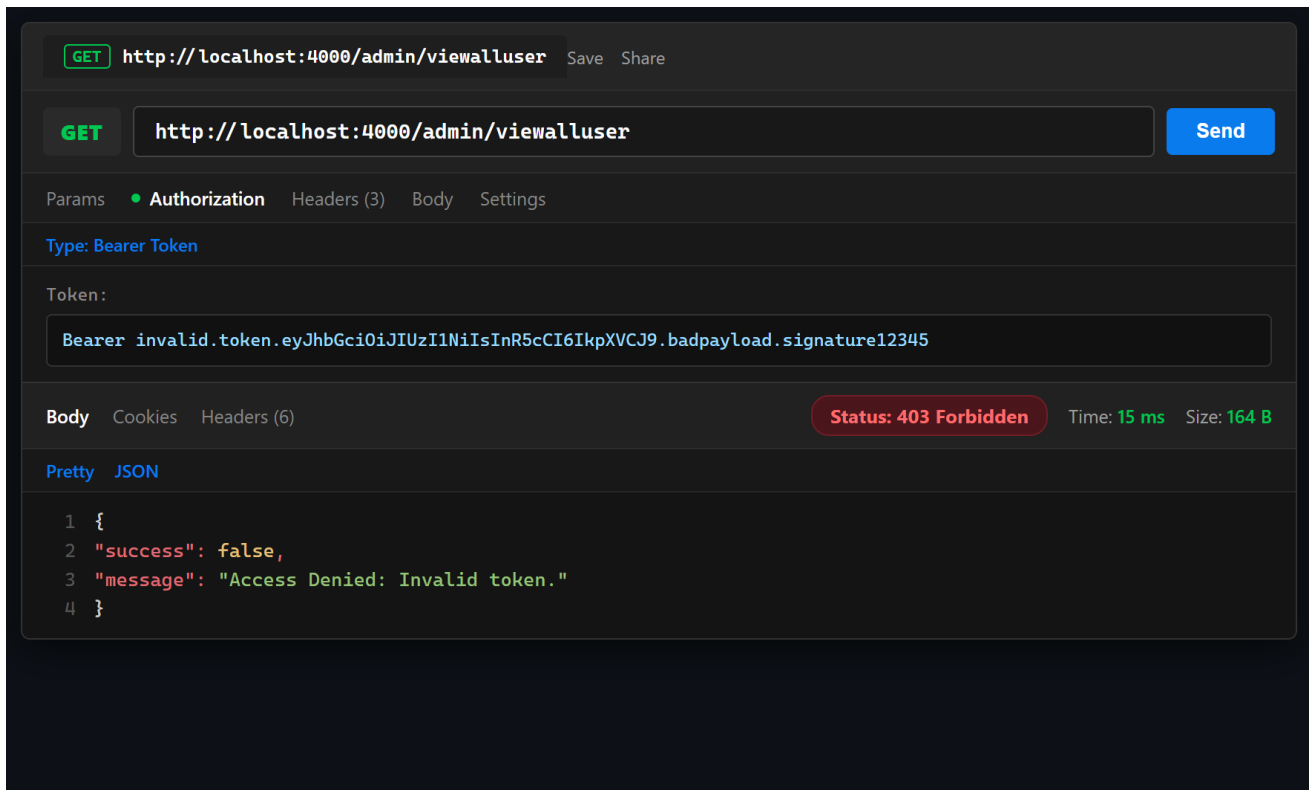
Security Test 10(a) – Access Admin API Without Token: Client attempts to access protected route GET

http://localhost:4000/admin/viewalluser without an Authorization header. API Gateway intercepts and immediately rejects the request with Status 401 Unauthorized ('Access Denied: No token provided').



Evidence 23: Postman - Protected endpoint accessed without Authorization header rejected with Status 401 Unauthorized (13ms).

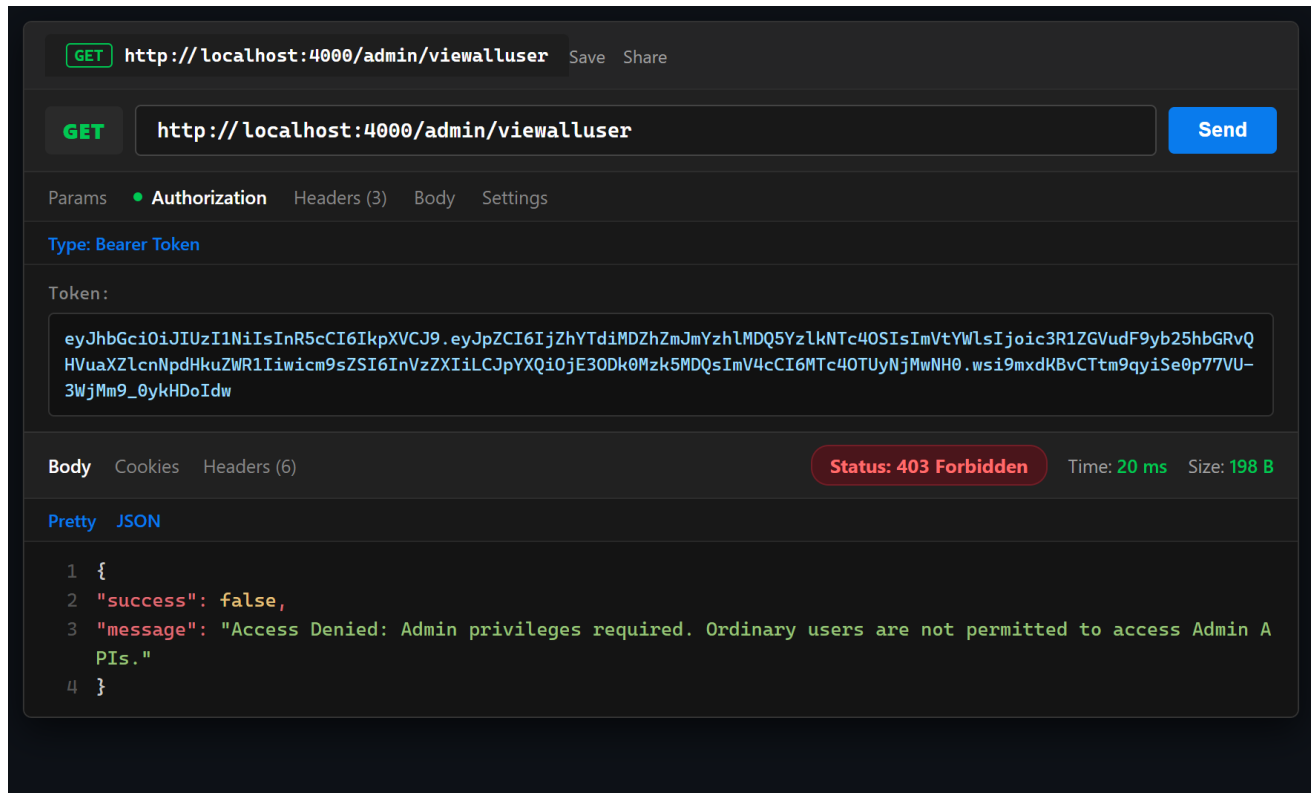
Security Test 10(b) – Access API Using Invalid JWT: Client attempts to access protected route using a fabricated or invalid token. API Gateway verifies token signature, detects invalid token, and rejects with Status 403 Forbidden ('Access Denied: Invalid token').



Security Test 10(c) – Admin Token Attempting to Access User API: Client uses an Administrator JWT token to access the User-only endpoint GET `http://localhost:4000/user/viewprofile`. API Gateway enforces role mutual exclusion and rejects the request with Status 403 Forbidden ('Access Denied: User privileges required. Administrators are not permitted to access User APIs').

Evidence 25: Postman - Administrator token attempting to access User API rejected with Status 403 Forbidden (21ms).

Security Test 10(d) – User Token Attempting to Access Admin API: Client uses an Ordinary User (Student) JWT token to access the Administrative endpoint GET `http://localhost:4000/admin/viewalluser`. API Gateway enforces role mutual exclusion and rejects the request with Status 403 Forbidden ('Access Denied: Admin privileges required. Ordinary users are not permitted to access Admin APIs').



Evidence 26: Postman - Ordinary user token attempting to access Admin API rejected with Status 403 Forbidden (20ms).

Task 11 – Public GitHub Repository

Public Repository Link: The complete source code including all 5 microservices, schemas, environment templates, and automated test scripts is published in a public GitHub repository. Sensitive files (.env, node_modules) are properly excluded via .gitignore.

Public GitHub Repository URL: https://github.com/goatronaldo/cld_assignment1_usermanagement.git

goatronaldo / cld_assignment1_usermanagement		Public	Branch: main
goatronaldo feat: complete Role-Based User Management System using Microservices API Gateway NodeJS MongoDB (Tasks 1-11)			
📁 APIGateway_Microservice	Add API Gateway with JWT verification and RBAC routing	Just now	
📁 Registration_Microservice	Add user registration with bcrypt password hashing and unique email	Just now	
📁 Login_Microservice	Add login authentication and 24h JWT token signing	Just now	
📁 Admin_Microservice	Add admin user search, view all users, and delete user APIs	Just now	
📁 User_Microservice	Add view own profile and update personal profile APIs	Just now	
📄 .env.example	Add environment variables template	Just now	
📄 .gitignore	Exclude node_modules and sensitive .env credentials	Just now	
📄 README.md	Comprehensive documentation, architecture diagram, and test guide	Just now	
📄 start_all.js	Orchestrator script to run all 5 microservices simultaneously	Just now	
📄 test_suite.js	End-to-end automated test suite for all assignment tasks	Just now	

Evidence 27: GitHub Web View - Public repository goatronaldo/cld_assignment1_usermanagement showing the 5 microservices and project files.